Pulp Riot Demand Forecasting Engine

Build Specification **v1.1** (supersedes v1.0) · For the build team · Source of truth for the demand module

Values marked DEFAULT are starting points to tune by backtest, not facts. Values marked ASSUMED are not in the case. All prototype data is synthetic, seeded from the case, and must be tagged SYNTHETIC in the UI. Green boxes mark what changed from v1.0; the full list is in section 14.

0. Goal

Build a Python engine and a thin UI that, for any as-of date, returns demand forecasts (P10/P50/P90) for 1, 2 and 4 weeks, a 6-8 week procurement outlook **and a season outlook for next summer (March-June)**, by flavor, channel, region and depot. It must also:

- detect demand shifts early, **up and down**, in quick commerce (QC) daily data and in weekly depot outflow for general and modern trade (GT/MT);
- size spikes with a scenario mixture, **including spikes on newly launched flavors**;
- adjust for stockout-censored sales and **report the demand lost to stockouts**;
- forecast new launches from analogs; keep all aggregates consistent;
- tag every number with its provenance and its uncertainty type.

The Inventory module reads the output through get_demand_for_inventory() (section 7), which returns **lead-time totals and sample paths**, not daily quantiles.

1. Architecture (two clocks)

```
monthly sales ------> [M2 monthly baseline] ---------------------------+
QC daily feed ------> [M3 daily QC filter] --> [M4 detector + scenario mixture]
  (orders_on_company first)                          ^          |
launch calendar ----> [M6 cold start] ------------+ (launch  |
                          |                         baseline) v
weekly depot stock -> [M1 validation + censoring] --> [M4b weekly depot-depletion detector]
  + receipts              |                                    |
                          +--------> [M5 temporal combine] <--+
                                            |
                      [M7 depot split + path aggregation] --> [M8 uncertainty] --> outputs
                                                                    (incl. SEASON horizon, lost demand)
```

The daily layer exists for QC only. GT and MT never get daily forecasts from data: they get monthly numbers phased across calendar days and tagged PHASED. Their early-warning signal is weekly depot outflow (M4b).

> **v1.1 change.** Added M4b (weekly depot-depletion detector) so GT/MT, 80% of volume, are no longer blind between month-ends. M6 now feeds M4 a baseline for launch series. M1 feeds censoring-adjusted demand to M7 for share estimation.

2. Red lines (do not violate, even if it looks simpler)

#	Rule
1	Do not **hardcode** a QC-to-GT/MT pass-through or lag. It may be **estimated** from data (M4b, section 6) and tagged ESTIMATED; until then the manual slider (default 0 = OFF) tagged ASSUMED is the only source. [v1.1: estimation now allowed]
2	Never add quantiles to aggregate - not inside the engine and **not across days at the Inventory interface**. Aggregate sample paths. The sum of P90s is not the P90 of the sum, and medians are not additive either.
3	No per-series seasonal estimation. Seasonality is pooled by group with shrinkage, **built only from uncensored months** (M2).
4	No fitted decay law (half-life fit) for spikes. Use the scenario mixture (M4).
5	Sales on a stockout-flagged period are a lower bound, not demand. Never learn a downward level move from a flagged observation. Never impute demand as max(sales, run-rate). **Detect on the uncensored order signal where it exists.**

#	Rule
6	No fixed region-to-depot shares. Shares are re-estimated **from censoring-adjusted demand, not raw outflow** (M7).
7	Always show the uncertainty type: SCENARIO (uncalibrated) or CONFORMAL (calibrated). The case's 30% error is a planning scale only.
8	Accuracy claims come only from the backtest harness (M9). **Conformal intervals use the engine's own backtest errors only, never the company's archived forecasts.**
9	Daily forecasts only for QC. Every other grain is PHASED.
10	The module forecasts demand, not fulfilment. Do not cap forecasts at stock or capacity inside this module.

3. Stack and repo layout

Python 3.11, pandas, numpy, scipy, DuckDB or SQLite, Streamlit for UI, pytest. No deep learning, no GPU, no external APIs. statsmodels optional.

```
pulpriot/
  config/params.yaml        # section 5
  data/generate.py          # M0 synthetic generator (section 8)
  data/schema.py            # tables + validation
  engine/censor.py          # M1
  engine/monthly.py         # M2
  engine/qc_daily.py        # M3
  engine/events.py          # M4 (upper + lower CUSUM, scenario mixture)
  engine/depletion.py       # M4b weekly depot-depletion detector + pass-through estimate   [v1.1]
  engine/combine.py         # M5
  engine/launch.py          # M6
  engine/hierarchy.py       # M7
  engine/uncertainty.py     # M8
  engine/lost_demand.py     # lost-demand estimate (section 6, M8)                        [v1.1]
  eval/backtest.py  eval/inject.py  eval/replay.py                                        # M9, section 10
  app/                      # Streamlit pages
  tests/
```

4. Data tables

Every fact table has a provenance column: OBSERVED, SYNTHETIC, PROXY or ASSUMED. Every input table carries an arrival column (arrival_date, report_received_ts, available_from or known_from); the engine may only use a row after it has arrived.

Table	Columns	Grain
dim_flavor	flavor_id, name, aliases, ingredients, analog_ids, analog_similarity, launch_date	flavor
dim_depot	depot_id, name, region_id	depot
fact_sales_monthly	month, flavor_id, depot_id, channel (nullable), units, value, available_from	monthly
fact_qc_daily	date, flavor_id, region_id, units_sold, **orders_on_company**, oos_flag (0/1/null), arrival_date	daily
fact_depot_stock_weekly	week_end, depot_id, flavor_id, closing_units, report_received_ts	weekly
fact_receipts_weekly	week_end, depot_id, flavor_id, opening_units, receipts, transfers_in, transfers_out, **implied_outflow, alloc_capped** [v1.1: required for M4b; was optional]	weekly
fact_forecast_archive	issue_month, issued_on, target_month, flavor_id, depot_id, forecast_units, **forecaster = COMPANY**	monthly vintage
fact_events	event_id, date, end_date, flavor_id, type (launch/promo/influencer/price/supply_disruption/competitor), planned, **known_from**, note	event
fact_stockout_log (optional)	date, depot_id, flavor_id, channel, days_out, reported_on	daily
fact_launch_plan [v1.1]	flavor_id, launch_week, planned_units (marketing plan), template_index	weekly
config_params	key, value, source (CASE/DEFAULT/ASSUMED/ESTIMATED)	key

Output tables:

Table	Columns
out_forecast	asof, flavor, channel, region, depot, horizon (7/14/28 d, 1-3 m, 8 wk, **SEASON**), p10, p50, p90, mean, interval_type (SCENARIO/CONFORMAL/EVENT_SCENARIO), grain_tag (OBSERVED_GRAIN/PHASED/**RECONCILED**), provenance_tag, extrapolating, censor_adjusted, lower_bound_only, model_version
out_forecast_daily	asof, flavor, channel, region, date, p10, p50, p90, grain_tag
out_paths [v1.1]	asof, flavor, channel, region, depot, path_id, day, units (sample paths for Inventory)
out_event	asof, flavor, region, **direction (UP/DOWN)**, **signal (ORDERS/UNITS_SOLD/DEPOT_OUTFLOW)**, stage, cusum_S, ratio_R, tier, weights per scenario, extra_bottles 7/14/28 d (p10/p50/p90), revenue_at_risk
out_lost_demand [v1.1]	asof, flavor, channel, region, depot, period, lost_p10, lost_p50, lost_p90, censored_days, revenue_lost
out_alert	asof, flavor, region, tier, message (plain language), recommended_action, data_freshness, approval_status
out_backtest	origin, horizon, level, model, metric, value, n

5. Config defaults (params.yaml)

Rows marked v1.1 are new or changed; the previous value is in brackets.

Key	DEFAULT	Meaning	Status
kappa	1	log offset so zeros work: z = log(y + kappa)	
cusum_k	0.5	slack, in sigma units (upper and lower)	
cusum_h	6.5	alarm threshold (TUNE with injection test). h=5 gives about 2.7 false alarms per summer over 20 series	v1.1 (5.0)
cusum_h_watch	3.5	freeze-baseline threshold. 2.5 froze baselines about 36 times per summer	v1.1 (2.5)
cusum_lower_h	6.5	threshold for the downward CUSUM	v1.1 new
detect_signal	orders_on_company	QC detection signal; fall back to units_sold only if orders are missing	v1.1 new
unfreeze_days	3	days with S = 0 before a WATCH state clears	
sustained_rebase_days	14	days in SUSTAINED before the level is rebased	v1.1 new
sigma_e_window	56	days for the robust noise estimate	
sigma_e_floor	0.03	minimum noise, mature series	
launch_age_days	56	series younger than this use launch rules	v1.1 new
sigma_e_floor_launch	0.15	minimum noise, launch series	v1.1 new
qc_level_gain	0.15	EWMA gain for the QC level	
weekday_min_days / weekday_shrink	56 / 4	clean days before weekday effects; kappa_w	
seasonal_shrink	0 / 0.25 / 0.5	weight on own profile, history under 24 / 24-35 / 36+ months	
seasonal_mask_censored	true	exclude censored months from seasonal profiles	v1.1 new
alpha_grid	0.05-0.60 step 0.05	pooled smoothing grid	
ratio_tiers	1.5 / 2 / 3	WATCH / ALERT / CRITICAL (DOWN: 1/1.5, 1/2, 1/3)	v1.1 (down added)
stage_eps	0.02	log-growth per day threshold for stage logic	
scen_half_lives / scen_prior	2,4,8 d / F1 1/9 each, F2 1/3, F3 1/3	scenario variants and prior weights	
scen_rmax_mult / scen_temper	1.5 / 0.5	F3 saturation cap; likelihood power	
scen_floor	0.05	minimum weight, applied with w = floor + (1 - K*floor) * w_norm	v1.1 (rule)

Key	DEFAULT	Meaning	Status
scen_exclude_oos_days	true	stockout days do not enter the weight likelihood	v1.1 new
event_min_post_days	4	post-peak days before weights move	
scenario_sigma0	0.376	planning scale: 0.30/0.798. Used as a log sigma, it implies a MAPE of about 31.5%	v1.1 (note)
scenario_grain_factor	depot 1.0 / region 0.8 / national 0.6	scenario-mode sigma by level (ASSUMED until conformal)	v1.1 new
conformal_min_n / decay_half_life	30 / 12 origins	pairs per cell; weight decay	
conformal_source	engine_backtest	never fact_forecast_archive	v1.1 new
dirichlet_alpha_weeks	4	prior share counts as 4 weeks of demand	
share_half_life_weeks	6	time decay on observed shares (replaces the n_w <= 4 cap)	v1.1 new
dirichlet_conc_cap	200	cap on Dirichlet concentration	
common_shock_rho / n_paths	0.5 / 1000	common variance share; Monte Carlo paths	
censor_cover_red / amber	1 / 3 days	cover thresholds for censoring	
censored_fraction_limit	0.30	above this, output is lower_bound_only	
depletion_k / depletion_h	0.5 / 4.0	weekly depot-depletion CUSUM (tune)	v1.1 new
pass_through_rho / mode	0 / ESTIMATED after 8 weeks	manual slider default; estimated once 8 weeks of overlap exist	v1.1 (mode)
price / margin	60 / 0.32	rupees per bottle; contribution margin	
lost_sales_definition	realised	8% of realised summer revenue = 0.768 cr bottles (alt. latent = 0.835 cr). ASSUMED	v1.1 new

6. Modules

M1. Validation and censoring flags (engine/censor.py)

validate(snapshot): flag duplicate keys, flavor-name mismatches (map through aliases), bad dates, negative or impossible stock, and stale sources (QC older than 2 days, depot report older than 8 days, monthly sales older than 40 days). Return a data_quality dict.

```
censor_prob(depot, flavor, week):
    cover_days = closing_units / recent_daily_outflow
    # recent_daily_outflow = mean of last 4 weeks of implied_outflow / 7 (receipts now required),
    #                        else last p50 forecast / 7
    p = 1.0 if cover_days < 1 else 0.5 if cover_days < 3 else 0.0
    p = 1.0 if a stockout_log row exists, or alloc_capped = 1 and cover_days < 7      # [v1.1]
    for QC days: p = oos_flag if present
```

How p is used (unchanged): p >= 0.5 and observed < p50 is MISSING (no level update, tag censored); p >= 0.5 and observed >= p50 is a normal update; weekly grain with 0 < p < 1 is MISSING plus DATA_QUALITY; censored share above censored_fraction_limit tags outputs lower_bound_only.

Censoring-adjusted demand (new output, used by M2 masks, M7 shares and lost demand):

```
d_adj[cell, period] = observed                    if p < 0.5
                    = max(observed, p50_forecast)   if p >= 0.5   # point estimate, tagged CENSOR_ADJUSTED
# d_adj is never fed back as a level observation (red line 5). It is used only for shares and loss estimates.
```

> **v1.1 change.** alloc_capped is tied to low cover so a capped order with ample stock is not treated as censored. receipts are required so implied_outflow exists for M4b.

M2. Monthly baseline (engine/monthly.py)

Pooled local-level model on the log scale: seasonal indices plus exponential smoothing.

```
1. Series s = (flavor, channel, depot): z[s,m] = log(units + kappa). Missing channel -> channel = ALL.
2. Group g = (flavor, channel). Seasonal profile: for each month-of-year j, mean over the group's series
   and years of (z - series mean), USING ONLY MONTHS WITH censored share < 0.5.                    [v1.1]
   If a month-of-year has no clean observation, fall back to the next level up (flavor, ALL),
   then to the portfolio profile. Centre to sum to zero.
3. gamma[s,j] = gamma[g,j] + lam_s * (raw[s,j] - gamma[g,j]),  lam_s from seasonal_shrink.
4. d = z - gamma.  l[t] = l[t-1] + alpha_g * (d[t] - l[t-1]); censored-down months skip the update.
   alpha_g per GROUP from alpha_grid, minimising pooled one-step absolute error after a 6-month burn-in.
5. Median   = exp(l[T] + gamma[s, moy(T+h)]) - kappa
   sigma_h  = sigma_1 * sqrt(h)
   Mean     = (median + kappa) * exp(sigma_h^2 / 2) - kappa                                       [v1.1]
   Intervals do NOT come from here (M8).
6. Under 6 months of data -> M6 (cold start).
7. Fallback benchmark seasonal_naive_growth: rho_s = (sum last 3 months)/(same 3 months a year earlier),
   rho = (3*rho_s + 3*rho_g)/6, forecast = y[m+h-12] * rho  (h <= 12; for h > 12 use y[m+h-24]*rho^2).
   Censored months in the numerator or denominator are replaced by d_adj.                        [v1.1]
8. Under 24 months of history: fallback is primary and tagged.
9. Consistency: aggregate summer share of annual units within 0.48 +/- 0.03, else DATA_CONFLICT.
10. SEASON horizon: run steps 5 or 7 for each month Mar-Jun of next summer and sum PATHS (M7), not P50s. [v1.1]
```

v1.1 change. Seasonality no longer learns from stockout months (they biased Punch and Dragon Passion low). Mean formula corrected for kappa. SEASON horizon added for the 8-month pre-build decisions (pulp, imports, co-packer).

M3. Daily QC filter (engine/qc_daily.py)

Run per (flavor, region), or per flavor nationally if the feed has no region. The input series is **q = orders_on_company** when present, else units_sold (tagged UNITS_SOLD_SIGNAL, and censored days are skipped).

```
z[d] = log(q[d] + kappa)
s[d] = monthly seasonal gamma for (flavor, QC), linearly interpolated between month midpoints
w[j] = weekday effect after weekday_min_days clean days:
       raw[j] = mean over clean days of (z - a - s) by weekday j
       w[j]   = n_j/(n_j + weekday_shrink) * (raw[j] - mean(raw))      else 0
resid = z[d] - s[d] - w[dow] - a[d-1]
a[d]  = a[d-1] + G * resid       G = qc_level_gain; G = 0 when frozen (M4) or censored-down (M1)
sigma_e = 1.4826 * MAD of in-control resid over sigma_e_window days, floored at sigma_e_floor
baseline forecast for day T+j: median exp(a[T] + s[T+j] + w[dow]) - kappa

LAUNCH SERIES (age < launch_age_days)                                                          [v1.1]
  a[d]    = log(M6 daily posterior P_d(d) + kappa) - s[d]      # baseline comes from the launch curve
  sigma_e = max(1.4826 * MAD of available resid, sigma_e_floor_launch)
  s[d]    = analog group seasonality (flavor's analogs, QC)
```

v1.1 change. A flavor in its first 8 weeks has no noise window, no seasonality and an EWMA that chases the normal launch ramp. It now uses the M6 launch curve as its baseline, so the normal ramp does not alarm and a viral spike on top of it does.

M4. Spike and drop detection, event scenarios (engine/events.py)

Per (flavor, region): state machine NORMAL, WATCH, ONSET, PEAK, DECAY or SUSTAINED, REVERTED. **Two one-sided CUSUMs run in parallel**: UP for spikes, DOWN for drops.

```
baseline b[d]:  NORMAL -> a[d-1];  WATCH/ONSET/... -> a_frozen;  launch series -> M6 posterior (M3)
zeta[d] = (z[d] - s[d] - w[dow] - b[d]) / sigma_e
S_up[d] = max(0, S_up[d-1] + zeta[d] - k)
S_dn[d] = max(0, S_dn[d-1] - zeta[d] - k)                                                      [v1.1]
NORMAL and S_up >= h_watch (or S_dn >= h_watch)  -> WATCH(direction); a_frozen = a[d-1]
S_up >= h  -> ONSET, direction UP (event open);  S_dn >= cusum_lower_h -> ONSET, direction DOWN
WATCH and S = 0 for unfreeze_days                -> NORMAL (unfreeze)
```

Ratio to the frozen baseline: R[d] = q[d] / exp(a_frozen + s[d] + w[dow]). UP tiers: WATCH if S_up >= h_watch or R >= 1.5; ALERT if an event is open or R >= 2; CRITICAL if R >= 3. DOWN tiers mirror these at 1/1.5, 1/2, 1/3. A feed lag of 1 day applies at detection time.

Stage logic: g[d] = OLS slope of log R over the last 5 days. ONSET until g < stage_eps for 3 days (then PEAK). After PEAK: g < -stage_eps gives DECAY; |g| < stage_eps for 7 days with R > 1.5 gives SUSTAINED; R inside exp(±2 sigma_e) of 1 for 5 days gives REVERTED (close event, unfreeze, restart level as the mean of the last 5 days of **deseasonalised, de-weekdayed** z). A drop while the stockout flag is on is not evidence of DECAY.

```
SUSTAINED rebase                                                                    [v1.1]
  after sustained_rebase_days in SUSTAINED:
     a[d] = mean over last 7 days of (z - s - w);  unfreeze;  close event as SUSTAINED_SHIFT
     write the shift to M2 as a level change for (flavor, QC, region) so the monthly layer learns it
```

Scenario mixture (unchanged shapes):

```
R_p = peak ratio (3-day mean at PEAK). During ONSET, R_p = latest 2-day mean R.
F1(tau), tau in {2,4,8}: R_j = 1 + (R_p - 1) * 0.5^(j/tau)                  # transient
F2:                      R_j = R_p                                           # plateau
F3:                      R_j = R_p + (R_max - R_p) * (1 - 0.5^(j/7))         # saturating growth
   R_max = min(R_p * exp(14 g), scen_rmax_mult * R_p) during ONSET, else scen_rmax_mult * R_p
prior: F1 variants 1/9 each, F2 1/3, F3 1/3
```

Weight update (only after event_min_post_days past the peak; before that weights stay at the prior and output is tagged EVENT SHAPE UNKNOWN):

```
days used  = post-peak days with oos_flag != 1 and M1 p < 0.5                        [v1.1]
loglik_k   = sum over used days of log N( log R[d] ; log m_k(d), sigma_R^2 ),  sigma_R = max(sigma_e, 0.05)
u_k        = prior_k * exp(scen_temper * loglik_k);   w_norm_k = u_k / sum(u)
w_k        = scen_floor + (1 - K * scen_floor) * w_norm_k        # K = 5 scenarios    [v1.1]
             -> every w_k >= 0.05 and sum(w) = 1 exactly (requires K * floor < 1)
```

Event impact: draw scenario k with probability w_k, apply daily lognormal noise exp(sigma_e * e - sigma_e^2/2) (mean 1), compute extra bottles over 7/14/28 days = sum over days of (R_j * noise - 1) * baseline_j, 2,000 draws, report P10/P50/P90. revenue_at_risk = extra bottles x price (also show x margin). For DOWN events, the same machinery reports **bottles at risk of overstock**.
Event-adjusted QC paths (baseline x R) feed M5 and M7.

The QC-to-GT/MT effect is never hardcoded. It is applied only as rho x (R - 1) x GT/MT baseline share when rho comes from the user slider (ASSUMED) or from the M4b estimate (ESTIMATED).

> **v1.1 change.** Fixed the weight floor: v1.0's 'floor then renormalise' pushed floored weights below 0.05 (e.g. 0.042), so its own test could never pass. Added the downward CUSUM for drops (rival launch, regional slowdowns behind the write-offs), detection on orders, stockout days excluded from the likelihood, and the SUSTAINED rebase.

M4b. Weekly depot-depletion detector (engine/depletion.py) [new in v1.1]

Gives GT/MT an early signal without outlet data. Runs per (flavor, depot) on implied weekly outflow.

```
y[w]     = implied_outflow[w] = opening + receipts + transfers_in - transfers_out - closing
expected = phased M2 P50 for that depot-week (all channels except QC, or ALL if channel unknown)
zeta[w]  = (log(y + kappa) - log(expected + kappa)) / sigma_w
           # sigma_w = robust std of the last 26 weeks, floor 0.08
If the week is censored (M1 p >= 0.5): skip the DOWN update; UP update uses y as a lower bound (allowed).
S_up, S_dn as in M4 with k = depletion_k, h = depletion_h
Alert also when cover_days = closing / (y/7) < 7 and S_up >= h_watch  ->  'depot running down faster than plan'

Pass-through estimate (replaces the permanent OFF switch):
  after >= 8 weeks of overlap, regress log(y_GT+MT / expected) on lagged log R_QC (lags 0-3 weeks), region-pooled,
  non-negative coefficients; rho_hat = sum of coefficients, tagged ESTIMATED with its standard error.
  rho_hat is used only if its 80% interval excludes 0; otherwise the slider value (default 0) applies.
```

M5. Temporal combine (engine/combine.py)

For QC series and each horizon H in {7, 14, 28} days, combine the daily layer (M3/M4) and the monthly layer (M2) in log space by inverse variance.

```
x_D = log(median of daily-path totals)     v_D = max(variance of log totals across paths, 0.05^2)
x_M = log(T_M)                             v_M = scenario_sigma(H, level)^2   (archive-free; see M8)
T_M = sum over days in the horizon of (monthly p50 / days in that month)
event open for this node: v_M = 10 * v_M
x = (x_D/v_D + x_M/v_M) / (1/v_D + 1/v_M)        T_H = exp(x)

Nested rescale (days must add up across horizons)                                   [v1.1]
  enforce T_14 >= T_7 and T_28 >= T_14 (raise the later total if needed)
  days 1-7   scaled by T_7 / D(1..7)
  days 8-14  scaled by (T_14 - T_7) / D(8..14)
  days 15-28 scaled by (T_28 - T_14) / D(15..28)        # D(a..b) = median-path sum over those days
  apply the same factors to every path, so path sums stay coherent
GT and MT: T = monthly layer total, daily values = T / days, tagged PHASED.
```

v1.1 change. v1.0 rescaled the 7, 14 and 28-day totals independently, so the first 7 days of the 14-day path did not equal the 7-day total. v_M now uses the horizon- and grain-scaled scenario sigma.

M6. New launch cold start (engine/launch.py)

Used for any flavor with under 6 months of history (Dragon Passion at launch). Inputs: analogs with similarity scores, a launch shape and a scale.

```
similarity s_i in (0,1] = mean of five manual 0-1 scores: ingredient overlap, price point, channel mix,
                          launch season, promo intensity
analog curve A_i(w)     = analog's own first 12 weeks, IF that launch is inside the data window;          [v1.1]
                          ELSE template(w) x analog run-rate in the same season last year
                          template(w) = generic launch shape, week-4 = 1 (fact_launch_plan, ASSUMED)
scale                   = marketing planned week-4 units (fact_launch_plan); DEFAULT analog week-4 units
P(w)    = scale * sum_i(s_i * A_i(w)) / sum_i(s_i) / (weighted week-4 value)
sigma_p = sqrt(sigma_a^2 + sigma_b^2 / mean(s))       sigma_a = 0.30, sigma_b = 0.69 until >= 3 launches exist
after n observed weeks:  delta = mean_w( log obs(w) - log P(w) ),  sigma_obs = 0.25
tau_p = 1/sigma_p^2;  tau_o = 1/sigma_obs^2
shift = tau_o * n * delta / (tau_p + n * tau_o);   sd_post = 1/sqrt(tau_p + n * tau_o)
forecast curve = P(w) * exp(shift)

Daily posterior for M3/M4                                                                               [v1.1]
  P_d(d) = P(week(d)) * exp(shift) / 7 * weekday(d) * QC channel share * region share,
  with week boundaries linearly interpolated so the curve has no weekly steps
  'obs' uses orders_on_company where available; censored weeks are excluded from delta
```

First 4 weeks: CUSUM on weekly delta flags LAUNCH_OVER / LAUNCH_UNDER. The daily detector (M4 with the launch baseline) is now the primary spike alarm; the weekly flag is a confirmation. Dragon Passion uses two analogs (Dragonfruit Bloom and Passionfruit Punch). All launch outputs are tagged EXTRAPOLATED; the UI shows the analog list and similarity scores.

v1.1 change. Both Dragon Passion analogs launched before the data window, so their first-12-week curves do not exist; v1.1 builds them from a launch template and last season's run-rate. The daily posterior gives M4 a launch-aware baseline.

M7. Hierarchy, depot split and aggregation (engine/hierarchy.py)

Base nodes are flavor x channel x region (monthly, plus daily for QC). Depot forecasts are never a fixed fraction.

```
pi_obs[k]  = time-decayed share of CENSORING-ADJUSTED demand d_adj (M1):                                [v1.1]
             weights 0.5^(age_weeks / share_half_life_weeks) over the last 26 weeks
n_eff      = sum of those weights
pi_hat[k]  = ( alpha_w * pi_prior[k] + n_eff * pi_obs[k] ) / ( alpha_w + n_eff )
             alpha_w = dirichlet_alpha_weeks (4);  pi_prior = last year's share (d_adj basis)

Monte Carlo, n_paths draws:
  e_common[flavor] ~ N(0,1),  e_node ~ N(0,1)
  log y_path[node] = log p50[node] + sigma_node * ( sqrt(rho)*e_common + sqrt(1-rho)*e_node )
  nodes with an open event: draw one scenario per (flavor, region) and use its R path
  depot shares: pi ~ Dirichlet( min(dirichlet_conc_cap, 4 * n_eff + alpha_w) * pi_hat )
  depot path = base path * pi
  aggregates (flavor, channel, region, total, SEASON) = sums of PATHS; quantiles of the summed paths

Coherence                                                                                               [v1.1]
  every path: sum of depots = region exactly (shares sum to 1)
  means: sum of depot means = region mean within 1e-6
  P50s are NOT required to add. If a planner view needs additive P50s, scale depot P50s to the region P50
  and tag them grain_tag = RECONCILED.
```

v1.1 change. v1.0's test required depot P50s to sum to the region P50, which contradicts red line 2 (medians are not additive). Shares now come from censoring-adjusted demand, because raw outflow shares understate stocked-out depots and overstate overstocked ones. This was the case's root cause. Time decay replaces the n_w <= 4 cap, which kept shares at least 50% anchored to last year.

M8. Uncertainty (engine/uncertainty.py)

```
for each cell (horizon, level, flavor-channel group):
  pairs = ENGINE forecast-actual pairs from the backtest harness (M9) only                          [v1.1]
          (fact_forecast_archive is the COMPANY's forecast and is never used here)
  pairs on censored actuals are excluded
  n = number of pairs
  if n >= conformal_min_n:  interval_type = CONFORMAL
       e_i = log(y_i + kappa) - log(p50_i + kappa);  w_i = 0.5^(age_i / conformal_decay_half_life)
       P10 = (p50+kappa)*exp(weighted q10 of e) - kappa;  P90 = (p50+kappa)*exp(weighted q90 of e) - kappa
  else: interval_type = SCENARIO (uncalibrated)
       sigma = scenario_sigma0 * sqrt(h_months) * grain_factor(level)                               [v1.1]
       (h_months = horizon_days / 30.4; grain_factor depot 1.0, region 0.8, national 0.6)
       P10 = p50 * exp(-1.2816 * sigma),  P90 = p50 * exp(+1.2816 * sigma)
  event open for the node: interval_type = EVENT_SCENARIO (quantiles of the scenario mixture, M4)
```

Show monthly coverage = share of the last 50 uncensored actuals inside [P10, P90]. Do not apply the 30% as a plus-or-minus band: under a normal reading of a 30% MAPE, ±30% covers only about 58% of outcomes.

Lost demand (engine/lost_demand.py, new): for every censored day (QC) or week (depot), lost = forecast path - observed, floored at 0, summed by PATH over the period; report P10/P50/P90 and revenue_lost = lost x price. Uncensored periods contribute 0. This is the number the deck uses to quantify last summer's loss.

> **v1.1 change.** Conformal intervals were calibrated on the company's archived errors, i.e. the wrong forecaster. Scenario sigma now scales with horizon and level instead of one 0.376 everywhere.

M9. Backtest and detection tests (eval/)

backtest.py: rolling origin, expanding window, minimum 12 months of training, at least 12 origins per horizon. As-of snapshots respect arrival (QC +1 day, weekly stock +2-4 days, monthly sales +5 days); assert no row used arrived after the origin. Horizons: QC 7/14/28 days; GT/MT 1, 2, 3 months; 8 weeks; **SEASON (scored once per season, reported separately)**.

Models compared: engine, naive, seasonal naive, seasonal naive x growth, the archived company forecast, 7-day and 28-day moving average (QC). Metrics: MASE and RMSSE, WAPE at aggregate levels, scaled bias, pinball loss at 0.1/0.5/0.9, 80% coverage, interval score. No MAPE where actuals can be zero or tiny. Stratify only where a cell has at least 30 observations. Report ordinary, stockout-flagged and event periods separately; **score accuracy on uncensored actuals only** (censored actuals are lower bounds).

Proposed acceptance criteria (to be agreed): MASE <= 0.9 vs seasonal naive x growth at each horizon with a block-bootstrap 90% interval excluding 1.0; scaled bias within ±5% at flavor level over the last 12 origins; 80% coverage between 72% and 88% on at least 50 pairs per cell. **On 28 months of synthetic data these criteria validate code, not real-world accuracy; rerun them on company data before go-live.**

inject.py (detection test): inject multiplicative ramps (length 3, 7, 14, 21 days x peak 1.5, 2, 3, 4, 5, 25 repeats = 500) **and drops (0.8, 0.67, 0.5, same lengths)** into event-free QC series, **plus ramps injected into launch-ramp series**. Report median and P90 delay to first ALERT, miss rate and false alarms per 100 series-days. Choose cusum_h so the whole portfolio (about 20 series x 122 summer days) has at most about one false alarm per summer. **Also test on residuals with lag-1 autocorrelation 0.3**, since autocorrelated noise raises false alarms.

7. Function contract (what other modules call)

```
get_forecast(asof, flavor=None, channel=None, region=None, depot=None, horizon='7d')   -> out_forecast rows
     horizon in {'7d','14d','28d','1m','2m','3m','8w','SEASON'}                                  [v1.1]
get_daily_path(asof, flavor, channel, region=None, depot=None, days=28)              -> out_forecast_daily
get_spike_status(asof, flavor=None, region=None, direction=None)                    -> out_event rows
get_depot_depletion(asof, flavor=None, depot=None)                                   -> M4b alerts      [v1.1]
get_demand_for_inventory(asof, flavor, depot, channel, lead_time_days, review_days=7)               [v1.1]
     -> { 'lead_time_total': {p10, p50, p90, mean},   # quantiles of PATH sums over the lead time
          'protection_total': {p10, p50, p90, mean},  # lead time + review period
          'paths': array[n_paths, lead_time_days + review_days],
          tags: interval_type, provenance, lower_bound_only, data_freshness }
     Inventory must compute safety stock from these totals or paths, never by summing daily P90s.
get_lost_demand(asof, flavor=None, period=None)                                      -> out_lost_demand [v1.1]
get_data_quality(asof)           -> stale sources, censored share, conflicts
run_backtest(config)             -> out_backtest
run_replay(start, end)           -> replay table
```

The Ask-the-Agent bar maps plain questions onto these functions. Answers include the source tag and data freshness. Every recommended action goes to out_alert with approval_status = PENDING until a named stakeholder approves or rejects it.

8. Synthetic data generator (data/generate.py)

Period: March of year Y-2 to June of year Y (28 months), daily for QC. Label everything SYNTHETIC; fix the random seed.

CASE values (use exactly): annual revenue 1,200 cr; price 60, so annual volume 20 cr bottles; summer (Mar-Jun Y) 9.6 cr observed; off-season about 1.3 cr per month; flavor shares 38/20/16/14/12 of summer volume; channel shares 55/25/20; 9 depots in 4 regions; Dragon Passion launches early March Y with zero prior sales; QC ramp to 4x over 14 days starting 14 days after launch; lost sales = 8% of summer revenue.

```
Back-solve, do not read off the shares                                                       [v1.1]
  Dragon Passion's 12% is OBSERVED volume that already contains the launch ramp, the 4x spike and the
  stockouts. Its pre-spike baseline B must be solved so that
      sum over summer of min( B x launch_ramp x spike x pass-through , available supply ) = 12% x 9.6 cr
  Same for every flavor: latent levels are calibrated so OBSERVED (after stockouts) matches the case shares.
  v1.0's 1.92 cr x 12% / 122 = 18,885 per day is circular and makes the generator overshoot 12%.

Lost sales: 8% of realised summer revenue = 46.08 cr = 0.768 cr bottles (ASSUMED; the alternative,
  8% of latent demand, gives 0.835 cr). Latent summer = 9.6 + 0.768 = 10.368 cr bottles.
```

ASSUMED values (configurable, tagged ASSUMED): 6% annual growth; region mapping 3/2/2/2; depot shares drawn around equal; weekday multipliers Mon 0.95, Tue 0.95, Wed 0.97, Thu 1.00, Fri 1.05, Sat 1.15, Sun 1.10; QC log noise 0.10; selectable post-peak shape (plateau_then_decay default, transient, sustained); 3-week passionfruit cap (60% availability) on Dragon Passion and Punch in May; GT/MT pass-through of the spike (truth only, so the engine must learn it); planner awareness at launch + 28 days.

Mechanics:

- Generate latent demand D per day, node and channel. Include at least one unannounced **downward** shift (e.g. rival launch, a regional slowdown) and one regional mix shift, so the lower CUSUM and the share re-estimation are exercised. [v1.1]
- Simulate depots: weekly replenishment to about 21 days of **planned** cover, allocations reset monthly on last year's mix, a plant supply cap; observed O = min(D, available stock). Write receipts, implied_outflow and alloc_capped. [v1.1]
- QC: units_sold = observed; orders_on_company = latent QC demand x small noise (uncensored order signal); oos_flag = 1 with probability 0.9 when short. [v1.1]
- fact_forecast_archive = company forecast (last year's censored sales x growth x lognormal error), calibrated so MAPE vs actual sales is 30% ± 2%; tag forecaster = COMPANY.
- Truth tables (truth_latent_daily, truth_events, truth_depot_share) are written separately; the engine must not read them.
- Dirty-data toggles: duplicates, misspelled flavor names, bad dates, negative stock, stale reports.
- Reconciliation: observed summer within 0.5% of 9.6 cr; lost within 2% of 0.768 cr; flavor shares within 0.5 points; channel shares within 1 point.

9. Tests with expected numbers (tests/)

Test	Expected	v1.1
Case constants	annual volume 20 cr; summer index 0.48 x 365 / 122 = 1.436	
Dragon Passion QC baseline	back-solved from the generator (about 11,700 per day on the reference seed); test that observed DP summer = 12% of 9.6 cr ± 0.5%. The 18,885 / 56,656 constants are withdrawn	changed
Upper CUSUM, noise-free ramp R_j = 4^(j/14), frozen baseline, k = 0.5, h = 6.5	sigma_e 0.10: crosses day 4; sigma_e 0.25: crosses day 7	changed (h)
Lower CUSUM, noise-free step to 0.8x, k = 0.5, h = 6.5	sigma_e 0.10: day 4; sigma_e 0.25: day 17	new
Ratio rule on the same ramp	R >= 2 first on day 7; R >= 3 first on day 12	
False alarms, N(0,1) in control, k = 0.5	ARL0 about 900 days at h = 5 vs about 4,100 at h = 6.5; about 0.6 false alarms per summer over 20 series at h = 6.5	new
Error scale	0.30 / sqrt(2/pi) = 0.376; coverage of ±30% under normal error = 0.575 ± 0.01	
Scenario weight floor	raw weights [.001,.001,.001,.001,.996] give min weight 0.0500 and sum 1.0000 (v1.0 rule gave 0.0418)	changed
Aggregation	for two or more nodes with rho < 1, P90 of summed paths < sum of node P90s	
Coherence	depot shares sum to 1; depot paths sum to the region path exactly; depot means sum to the region mean within 1e-6. P50s are not tested for additivity	changed
Mean with kappa	median 10, kappa 1, sigma 0.5 gives 11.465 (not 11.331)	new
Temporal nesting	sum of days 1-7 = T_7 and sum of days 1-14 = T_14 exactly; T_14 >= T_7	new
Censoring	with p = 1: observation below p50 leaves the level unchanged; above p50 updates it	
Censored seasonality	a month with censored share >= 0.5 has zero weight in gamma	new
Launch baseline	launch ramp alone (no spike) raises no ALERT in the first 56 days; ramp + 4x spike raises ALERT	new
Detection signal	with orders_on_company present, units_sold is never used for zeta	new
Stage machine	event closes and baseline unfreezes after 5 days within exp(±2 sigma_e) of 1; SUSTAINED for 14 days rebases the level	changed
Scenario weights	unchanged from prior before 4 post-peak days; none below 0.05 afterwards; always sum to 1; OOS days excluded	changed
Temporal combine	v_M very large gives the daily total; equal variances give the geometric mean	
Cold start	n = 0 gives the prior curve exactly; large n converges to observed; sd_post decreases with n; analog curve built from template when the analog's launch is outside the data	changed
Depot shares	after a step change in true shares, pi_hat moves at least halfway to the new share within 8 weeks; censored depots are not under-weighted	new
Conformal source	no fact_forecast_archive rows enter M8	new
Inventory interface	lead-time P90 < sum of daily P90s whenever rho < 1	new
Lost demand	zero on uncensored periods; on synthetic data the replay's P10-P90 band contains the true lost total	new
No look-ahead	backtest asserts no arrival timestamp after the origin	
Provenance	every output row has non-null provenance_tag, interval_type, grain_tag	

10. Replay of last summer (eval/replay)

Run the engine for every as-of date from launch - 7 to launch + 120 (through the highway closure and the rival launch). Output a timeline table and chart: latent demand, observed sales, orders signal, forecast P10/P50/P90, first WATCH / ALERT / CRITICAL dates (UP and DOWN), M4b depot alerts, event-impact estimates vs truth, and lost demand vs truth. Compare the first alert date with the planner-awareness date (DEFAULT launch + 28 days, ASSUMED). Run the replay twice, detecting on orders_on_company and on units_sold, to show the value of the uncensored signal. Repeat for each selectable true spike shape to show the scenario weights converging.

11. UI pages (Streamlit)

- **Forecast explorer:** as-of date, flavor, channel, region, depot, horizon (incl. SEASON). Table and fan chart. Badges: interval type, provenance, PHASED / OBSERVED_GRAIN / RECONCILED, EXTRAPOLATING, LOWER_BOUND_ONLY.
- **Spike and drop monitor:** daily QC ratio, S_up and S_dn, stage, scenario weights, extra or excess bottles and revenue at risk for 7/14/28 days, tier, signal used, data freshness.
- **Depot depletion [v1.1]:** M4b alerts by depot and flavor, cover days, estimated pass-through with its interval.
- **Data quality and censoring:** stale sources, censored share, flagged cells, DATA_CONFLICT.
- **Backtest report:** tables vs benchmarks with pass/fail.
- **Replay:** section 10, including lost demand vs truth.
- **Alerts and approvals [v1.1]:** every recommendation in plain language ('why this alert', the numbers behind it, the action), with approve/reject and a log.
- **Config and provenance:** edit params.yaml, show source tags. Revenue at 60 per bottle with a revenue/contribution toggle (margin 0.32).
- **Simple mode [v1.1]:** seasonal naive x growth plus ratio tiers, for planners who need an explainable fallback.

12. Work split and timeline (42 hours, 4 people)

Priorities. **P0:** M0, M1 (incl. d_adj), M2 (fallback + pooled, censor masks), M3 (incl. launch baseline), M4 (upper + lower CUSUM, fixed floor), M7 basic (paths, coherence on means), M8 scenario mode (scaled sigma), get_demand_for_inventory with totals and paths, replay, section 9 tests. **P1:** M4b depot depletion, M5 with nested rescale, M6, lost demand, conformal mode, injection test, SEASON horizon. **P2:** weekday effects, pass-through estimation, share-draw refinements. If time runs short, cut from the bottom and keep the interfaces.

Owner	Modules
A: data	M0 generator, schema, loaders, M1 validation, censoring and d_adj
B: core forecast	M2, M3 (incl. launch baseline), M5
C: events and launches	M4 (both CUSUMs), M4b, M6, injection test
D: aggregation, uncertainty, app	M7, M8, lost demand, M9 backtest, Inventory interface, Streamlit app

Hours	Milestone
0-4	Repo, schema, params.yaml v1.1, function stubs for every module, generator v1
4-14	Module v1 each; unit tests for each
14-24	End-to-end run for Dragon Passion first, then all flavors; UI skeleton
24-32	Backtest, replay (orders vs units_sold), injection test and tuning of cusum_h
32-38	All tests green, UI polish, provenance and interval badges everywhere
38-42	Demo rehearsal and freeze (no new features)

Definition of done: pytest passes; replay runs end to end; backtest report generates; every number in the UI carries its tags; no red line in section 2 is violated; the Inventory module consumes totals or paths, never summed daily quantiles.

13. Facts to confirm with the company (the real-world version)

- QC feed fields, grain (platform, store, region), history length, and whether planners can receive it next-day; **what orders_on_company is (dark-store purchase orders?) and whether it continues when stock is out**.
- What the 30% forecast error measures (MAPE, WAPE, median) and over how many periods; **whether '8% of summer revenue lost' is on realised or latent revenue**.
- Whether month-end sales are split by flavor, depot and channel, and whether they are dispatches or sales.
- Whether depots can report **receipts and transfers weekly** (needed for implied outflow), and whether a stockout log exists.
- Whether the QC flavor mix matches the overall flavor mix.
- Whether the 60-day shelf-life rejection applies to general trade.
- **Who approves which recommendation** (planner, regional head, COO) and the response time expected during peak.

14. Change log: v1.0 to v1.1

ID	Type	Where	v1.0 problem	v1.1 fix
E1	Error	M4	Floor-then-renormalise pushes floored weights below 0.05; test could not pass	w = floor + (1 - K floor) w_norm
E2	Error	M7, tests	Required depot P50s to add to region P50 (contradicts red line 2)	Coherence on paths and means; RECONCILED tag for additive P50s
E3	Error	M8	Conformal pooled the company's archived errors	Engine backtest errors only; archive tagged COMPANY
E4	Error	Section 8, tests	DP baseline 18,885 read off the observed 12%, which already includes spike and stockouts	Back-solved baseline; constants withdrawn
E5	Ambiguity	Section 8	'8% lost' realised vs latent undefined	Realised (0.768 cr), tagged ASSUMED; alternative stated
M1	Minor	M2	Mean ignored kappa	(median + kappa) exp(s^2/2) - kappa
M2	Minor	M5	Horizons rescaled independently; days did not nest	Cumulative piecewise rescale
M3	Minor	M8	One sigma for every horizon and grain	sigma0 x sqrt(h) x grain factor
M4	Minor	M8	0.376 is a linear-MAPE scale used as a log sigma	Documented (implied MAPE about 31.5%)
G1	Gap	M3, M4, M6	Spike on a 2-week-old launch: no baseline, noise or seasonality	M6 daily posterior as baseline; launch sigma floor 0.15
G2	Gap	M3, M4	Detection on stock-capped units_sold; orders column unused	Detect on orders_on_company; OOS days out of likelihood
G3	Gap	M4b	No early signal for GT/MT (80% of volume); pass-through OFF forever	Weekly depot-depletion CUSUM; estimated pass-through
G4	Gap	M4	Upward-only detection	Lower CUSUM and DOWN tiers
G5	Gap	Config	h = 5 gave about 3 false alarms per summer; WATCH 2.5 froze about 36 times	h = 6.5, h_watch = 3.5, autocorrelation test
G6	Gap	M4	SUSTAINED never rebased	Rebase after 14 days; feed M2
G7	Gap	M7	Shares anchored to last year; raw outflow censored	Censoring-adjusted, time-decayed shares
G8	Gap	M2	Seasonality learned from censored months	Censored months masked
G9	Gap	M6	Analog first-12-week curves outside the data window	Template x analog run-rate
G10	Gap	M2, M7, contract	No horizon beyond 3 months	SEASON horizon from path sums
G11	Gap	Contract	Inventory got daily quantiles, inviting summed P90s	Lead-time totals and paths
G12	Gap	M8	No lost-demand estimate	out_lost_demand from paths

ID	Type	Where	v1.0 problem	v1.1 fix
R1	Readiness	M9, section 13	Accuracy on synthetic data could be mistaken for real accuracy	Explicit: rerun acceptance on company data
R2	Readiness	UI, contract	No approval flow; hard to explain to 15 spreadsheet planners	Approval log, plain-language alerts, simple mode